WNDRVR
SUPPORT NETWORK

VXWORKS 7 ITL_ARRIA10 BSP SUPPLEMENT GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 itl_arria10 BSP Supplement Guide, SR0650

12 August 2020

1. INTRODUCTION

1. Introducing the itl_arria10 BSP

This document describes the features of the itl_arria10 BSP, which is designed to run on the Intel Arria 10 SoC development board.

This document must be read in conjunction with the *itl_arria10_readme.md* BSP readme file that relates to your hardware board.

While the BSP readme file guides you through a simple target boot process, this document gives a fuller description of the BSP and its capabilities.

Before using a board with VxWorks®, verify that the board runs in the factory configuration, and check all target board connections. Refer to the board vendor documentation for assistance.

2. Introducing the Boot Process

This BSP supports booting the target board from flash memory using U-Boot which then loads the VxWorks kernel.

Four approaches can be taken when booting the target board.

Table 1. Target Boot Strategies

Boot Loader	Image File Loaded By Boot Loader	Image File Data Model	Image File Name	Independent or Embedded DTB
U-Boot	VxWorks kernel	32-bit	uVxWorks	Independent or Embedded
U-Boot	VxWorks kernel	32-bit	vxWorks.bin	Embedded

3. Configure Development Environment

Initialise the workstation command shell for VxWorks Board Support Package (BSP) configuration and project build.

If your workstation runs Windows, refer to Setting Up a Windows Development Environment on page 1 for instructions.

If your workstation runs Linux, refer to Setting Up a Linux Development Environment on page 2 for instructions.

Setting Up a Windows Development Environment on page 1 The Windows command-line interface must be configured correctly for use with VxWorks.
Setting Up a Linux Development Environment on page 2 The Linux command-line interface must be configured correctly for use with VxWorks.

3.1. Setting Up a Windows Development Environment

The Windows command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
C:\> cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
C:\WindRiver> wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright © 2009 Microsoft Corporation. All rights reserved.

C:\WindRiver>
```

3. Change into the Workbench workspace directory.

```
C:\WindRiver> cd workspace
C:\WindRiver\workspace>
```

Parent topic: Configure Development Environment on page 1

3.2. Setting Up a Linux Development Environment

The Linux command-line interface must be configured correctly for use with VxWorks.

Before You Begin

Before you begin you must install Wind River VxWorks 7 on your host workstation.

About This Task

These steps configure the workstation command terminal environment variables to allow for VxWorks command-line interface (CLI) development tasks to be performed.

Procedure

1. Change into the Wind River directory.

```
~/home/user$ cd WindRiver
```

2. Add Wind River build tools to the command terminal path.

```
~/home/user/WindRiver$ ./wrenv.linux -p vxworks-7
~/home/user/WindRiver$
```

3. Create the workspace directory.

```
~/home/user/WindRiver$ mkdir workspace
~/home/user/WindRiver$
```

4. Change into the workspace directory.

```
~/home/user/WindRiver$ cd workspace
~/home/user/WindRiver/workspace$
```

Parent topic: Configure Development Environment on page 1

2. CONFIGURATION AND SETUP

1. Updating the U-Boot Image on SD/MMC Card

Contact the vendor for the SD card image and follow their instructions for updating the image on an SD or MMC card.

2. Device Tree and Device Tree Blob Files

This board support package (BSP) uses device tree nodes to describe the device specific information on the board. The supported drivers will get configuration data from the device tree source file (suffixed .dts) that is located in the BSP directory.

For a description of the device tree source file structure, refer to the device tree specification.

The Intel Arria 10 SoC development board is supported by this BSP, and the corresponding DTS file is **itl_arria10_devkit_ghrd.dts**.

The device tree source file in this BSP can be used in your VxWorks projects in either embedded or independent mode. In embedded mode, the device tree blob is embedded in the VxWorks image when the VIP is built. The VIP parameter DTS_FILE is set to point to the BSP DTS file in the BSP directory.

3. Workstation Serial Driver

Serial connectivity is supported on the Intel Arria board, and it is delivered through the board USB port. A USB2UART workstation driver must be installed before serial connectivity is available with the target board. Locate the workstation USB2UART driver from the FDT website, http://www.ftdichip.com/.

The serial communication parameters are:

- Baud Rate: 115200
- Data: 8 bit
- Parity: None
- Stop: 1 bit
- Flow Control: None

4. Prepare the U-Boot Environment

Refer to *itl_arria10_readme.md* for instructions on preparing the default U-Boot environment on the target board.

If you require to use an Early I/O feature then you must disable programming of the "core" part of the FPGA design by removing the "run core_rbf_prog" command. Assuming you have followed the procedure in the readme file above, do this from the U-Boot serial console as follows:

```
=> setenv vxbootimage vxWorks.bin
=> setenv vxloadaddr 0x100000
=> setenv vxmmcload 'mmc rescan;${mmcloadcmd} mmc 0:${mmcloadpart} ${vxloadaddr} ${vxbootimage}'
=> setenv bootcmd 'run vxmmcload;fpgabr 1;go ${vxloadaddr}'
=> saveenv
```

5. Boot With a non-U-Boot Bootloader

The Arria10 board can be booted using non-GPL bootloaders such as UEFI or MPL. Contact the board vendor for details.

3. VXWORKS PROJECTS

1. Project Creation Workflow

A VxWorks source build (VSB) project must always be created before the VxWorks image project (VIP).

When creating a VSB using the itl_arria10 BSP, Wind River recommends you also specify the name of the Board Support Package (BSP) at VSB creation time, as this automatically configures the VSB with the necessary VxWorks layers.

When creating the VSB, you have the option of specifying the CPU instead of the BSP. If you do this, you must remember to add the correct VxWorks layers to the VSB before building the project. Refer to Creating a 32-Bit CPU Based VxWorks Source Build on page 7 for details on how to create the VSB project and add VxWorks layers to it.

From the command-line, you specify the BSP in the VSB creation command by adding -bsp itl_arria10. You specify the 32-bit CPU in the VSB creation command by adding -cpu ARMARCH7 to the VSB creation command.

Note: 64-bit CPU support is not available on this target.

Note: The project creation workflow can also be achieved within the Workbench Eclipse IDE. For details, refer to *Workbench 4 Getting Started: Workbench Project Overview*

2. Processing Environments and Data Models

When building projects with this Board Support Package (BSP) you can choose between two 32-bit processing environments.

There are two variations supported by the BSP:

- ARM v7 - 32-bit arm CPU uniprocessor
- ARM v7 - 32-bit arm CPU symmetric multiprocessor

Note: All 32-bit arm projects are also referred to as using ARM v7, or ARMARCH7 build spec.

Table 1. Processing Environment and Data Model VSB Options

Data Model Option	Parameter Specified During VSB Creation
ARM v7 32-bit Data Width	-ilp32 (default)
Uniprocessor	-up
Symmetric Multiprocessor	-smp (default)

Note: When creating a VxWorks Source Build (VSB), if you do not specify -up or -smp, the project defaults to symmetric multiprocessor on creation.

3. Creating a Uniprocessor VxWorks Source Build

Before You Begin

Ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -up -bsp itl_arria10 -S myVSB
C:\WindRiver\workspace> cd myVSB
```

> **Note:** Creation of a symmetric multiprocessor VSB simply requires the omission of the option -up. For an example, refer to *nxp_ls1021a-pb_readme.md*, located in the BSP directory.

2. Build the VSB.

```
C:\WindRiver\workspace\myVSB> vxprj build
```

> **Note:** To build the VSB with parallelism enabled, you must use the command make. For example, if the workstation has eight cores, you build the project using all of these cores with the following command:
>
> ```
> C:\WindRiver\workspace\myVSB> make JOBS=8
> ```

4. Creating a 32-Bit CPU Based VxWorks Source Build

Before You Begin

Ensure your VxWorks development environment is correctly configured. Refer to Configure Development Environment on page 1.

Procedure

1. Create the VSB.

```
C:\WindRiver\workspace> vxprj vsb create -force -cpu ARMARCH7 -S myVSB
C:\WindRiver\workspace> cd myVSB
C:\WindRiver\workspace\myVSB> vxprj vsb add ITL_SOC_ARRIA10
```

2. Build the VSB.

```
C:\WindRiver\workspace\myVSB> vxprj build
```

Note: To build the VSB with parallelism enabled, you must use the command make. For example, if the workstation has eight cores, you build the project using all of these cores with the following command:

```
C:\WindRiver\workspace\myVSB> make JOBS=8
```

5. The VxWorks Image Project

Refer to *itl_arria10_readme.md* for instructions on creating a VxWorks image project (VIP) for your board.

Creating a VxWorks Kernel Image with Embedded DTB on page 8

5.1. Creating a VxWorks Kernel Image with Embedded DTB

Before You Begin

These steps assume you have already created a VxWorks source build (VSB) configured in either uniprocessor or symmetric multiprocessor configuration.

Procedure

1. Create the VxWorks Image Project (VIP).

```
C:\WindRiver\workspace> vxprj vip create -vsb myVSB -profile PROFILE_DEVELOPMENT itl_arria10 my
VIP
C:\WindRiver\workspace> cd myVIP
```

Note: If you do not specify the VIP creation parameter -profile PROFILE_DEVELOPMENT, then you must remember to manually include the network END driver component into the VIP, **INCLUDE_END**.

2. Select an embedded Device Tree Blob (DTB) file.

```
C:\WindRiver\workspace\myVIP> vxprj vip component add myVIP INCLUDE_STANDALONE_DTB
```

3. Configure the VxWorks boot line for the BSP. Refer to the BSP readme file for your board.
4. Set the Target Board MAC address in the Device Tree Source (DTS) file. This step is only required if you decide not to set the MAC address in the boardEndMacGet( ) function, as described in Adding the Ethernet Address in boardEndMacGet() on page 12.
   1. Edit the device tree source file *itl_arria10_devkit_ghrd.dts*.
   2. Locate the network device:

```
emac0: ethernet@FF800000
```

   3. Edit the **local-mac-address** property.

```
local-mac-address = [ 00 0A 35 11 22 33 ];
```

Note: Replace the specified MAC address with your target board MAC address.

5. Build the VIP.

WNDRVR

```
C:\WindRiver\workspace\myVIP> vxprj build
```

Parent topic: The VxWorks Image Project on page 8

4. TARGET BOOT WORKFLOWS

1. Downloading a Binary VxWorks Image With Embedded DTB from U-Boot

Before You Begin

These steps assume you have created and built a VxWorks image project (VIP) with embedded DTB. Refer to Creating a VxWorks Kernel Image with Embedded DTB on page 8 for more details. They also assume you have a TFTP server running on your workstation, configured to serve files from the VIP default directory.

Procedure

1. Power up the target board to the U-Boot shell.
2. Setup the target network parameters as follows:

```
=> setenv ipaddr 192.168.0.3
=> setenv netmask 255.255.255.0
=> setenv serverip 192.168.0.2
=> setenv gatewayip 192.168.0.1
=> setenv bootargs 'emac(0,0)host:vxWorks h=192.168.0.2 e=192.168.0.3:fffffe00 g=192.168.0.1 u=
target pw=vxTarget f=0x00'
=> saveenv
```

Note: Make sure you set the target network parameters appropriately for your network.

3. Load the VxWorks image into target memory.

```
=> tftp 0x100000 vxWorks.bin
```

4. Run the VxWorks kernel.

```
=> go 0x100000
```

2. Downloading a uVxWorks Image File With Embedded DTB from U-Boot

Before You Begin

These steps assume you have created and built a VxWorks image project (VIP) with embedded DTB. Refer to Creating a VxWorks Kernel Image with Embedded DTB on page 8 for more details. They also assume you have a TFTP server running on your workstation, configured to serve files from the VIP default directory.

Procedure

1. Power up the target board to the U-Boot shell.
2. Setup the target network parameters as follows:

```
=> setenv ipaddr 192.168.0.3
=> setenv netmask 255.255.255.0
=> setenv serverip 192.168.0.2
=> setenv gatewayip 192.168.0.1
=> setenv bootargs 'emac(0,0)host:vxWorks h=192.168.0.2 e=192.168.0.3:fffffe00 g=192.168.0.1 u=
target pw=vxTarget f=0x00'
=> saveenv
```

Note: Make sure you set the target network parameters appropriately for your network.

3. Load the VxWorks image into target memory.

```
=> tftp 0x1000000 uVxWorks
```

4. Run the VxWorks kernel.

```
=> bootm 0x1000000
```

5. DEVICE-SPECIFIC INFORMATION

1. L2 Cache

This BSP supports L2 cache by default from the VIP component **DRV_CACHE_FDT_PL310**.

2. Network and Ethernet (MAC) Configuration

The Gigabit ethernet MAC is implemented in the Intel SoC Arria10. The hardware supports 10mbs/100mbps/100mbps and only HPS-EMAC port is supported.

To enable the MEMAC controller, you can follow one of two methods:

1. Specify **PROFILE_DEVELOPMENT** when creating the VxWorks image project (VIP). The related components are added and the network is configured.
2. Create the VIP with the default profile and add the component **INCLUDE_END** to it.

Adding the Ethernet Address in boardEndMacGet() on page 12
Configuring the Ethernet Address from the U-Boot Console on page 13U-Boot will usually allow the target board ethernet (MAC) address to be updated from the U-Boot serial console.

2.1. Adding the Ethernet Address in boardEndMacGet()

About This Task

These steps assume you have created a VxWorks image project (VIP) called **myVIP**.

These steps declare a function, boardEndMacGet( ), which has four parameters defined as follows:

ifName

A pointer to a string defining the network interface name of the network controller.

ifUnit

The network interface index number.

ifMacAddr

A pointer to the memory storing the MAC address.

ifMacAddrLen

The length of the MAC address.

Procedure

1. Locate the source file sysLib.c( ).

   This is usually located in the workspace at **myVIP/***bsp_name_w_x_y_z***/sysLib.c**.

WNDRVR

> **Note:** The BSP layer version number is represented by *w_x_y_z*.

2. Identify and modify the structure **boardFuncTbl**.

   Modify the **boardFuncTbl** structure as follows:

```
BOARD_FUNC_TBL boardFuncTbl =
{
    ….
    /* .endMacGet = */ boardEndMacGet
```

3. Create a forward declaration for function boardEndMacGet( ).

```
/* forward declarations */

LOCAL STATUS boardEndMacGet
    (
    char *          ifName,
    unsigned int    ifUnit,
    UINT8 *         ifMacAddr,
    unsigned int    ifMacAddrLen,
    );
```

4. Create the function boardEndMacGet( ).

   Add the function to the bottom of the file.

```
STATUS boardEndMacGet
    (
    char *          ifName,
    unsigned int    ifUnit,
    UINT8 *         ifMacAddr,
    unsigned int    ifMacAddrLen
    )
    {
    UINT8 macAddr[6] = {0x00, 0x0A, 0x35, 0x11, 0x22, 0x33};

    if ((ifName == NULL) || (ifMacAddr == NULL))
        return ERROR;

    bcopy(macAddr, ifMacAddr, ifMacAddrLen);

    return OK;
    }
```

> **Note:** Update the array **macAddr** directly to reflect the actual ethernet address (MAC address) of your target board. Alternatively, code this function to retrieve the address from a memory device on the board and then set the array **macAddr** to it.

Parent topic: Network and Ethernet (MAC) Configuration on page 12

2.2. Configuring the Ethernet Address from the U-Boot Console

U-Boot will usually allow the target board ethernet (MAC) address to be updated from the U-Boot serial console.

Refer to your target documentation to check if U-Boot supports setting the *ethaddr* variable, and also whether you can save the environment variables from the U-Boot console. If it does then VxWorks will inherit the ethernet address you set from U-Boot, and this new address will persist beyond a reboot. Set the address as follows:

=> **`setenv ethaddr`** *AA:BB:CC:DD:EE:FF* **`;saveenv`**

Parent topic: Network and Ethernet (MAC) Configuration on page 12

3. Timer

Various hardware timers are provided in either the processor core or in the SOC.

General Purpose Timer (GPT)

These are the public timers in the SOC. They are added to the VxWorks image project (VIP) through the component **DRV_TIMER_FDT_SYNOPSYS_DW_TIMER** and can be used as both as the system clock in this BSP.

ARM Generic Timer (GTC)

This is the global timer in the ARM processor. It is supported by the **DRV_TIMER_FDT_ARM_GTC** component.

4. PL330 DMA

The SDMA is supported by this BSP and is included into the VIP with the component **INCLUDE_DMA_SYS**. The application then must use the VxBus DMA APIs to access the DMA function.

5. I2C

I2C is supported by this BSP by adding the component **DRV_SYNOPSYS_DW_I2C** to the VIP. The application must then use the VxBus API to access the I2C function.

6. RTC

The RTC is supported by this BSP and is included into the VIP with the component **DRV_I2C_RTC**. The application then must use the VxBus RTC APIs to access the RTC function. RTC alarm function is not supported because the RTC alarm interrupt is not connected to any interrupt controller or GPIO configured on the hardware.

6. KNOWN BSP ISSUES

1. SMP

VxWorks Symmetric Multiprocessing (SMP) is supported when this BSP is used on the supported boards. For details on VxWorks SMP, refer to the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

The main issues relating to SMP on this BSP are:

1. MMU
   MMU-less configuration is unsupported.
2. Cache and Bus Snooping
   Cache cannot be disabled in an SMP system. Direct calls to cacheDisable( ) do nothing.
   Only copy-back mode is tested with this BSP. Changing the configuration to write-through mode may result in unpredictable behaviour.
3. CPU Number
   The VIP parameter VX_SMP_NUM_CPUS specifies the number of CPUs assigned to the SMP kernel. The CPU referred to includes threads, hyper-threads and so forth.
   VxWorks supports using less than the total number of CPUs in an SMP system. This permits some CPUs to be used for additional tasks such as dedicated tasks or additional asymmetric operating systems.
   CPUs not assigned to the SMP kernel will remain in the default, power-on reset state.

2. U-Boot Issues

If the U-Boot image is not updated to U-Boot 2014.10 (Mar 23 2018 - 15:53:11), then the following U-Boot features will not work:

- uVxWorks image support
- Independent DTB file support

7. REFERENCES

1. See Also

- VxWorks 7 Platform Programmer's Guide
- VxWorks 7 Application Programmer's Guide
- VxWorks 7 Configuration and Build Guide
- VxWorks 7 ARM Architecture Guide
- VxWorks 7 BSP and Driver Guide